

Validate, fetch, branch

Three small R packages for reproducible work with biological web services

Samuel Bharti

biobouncer, biohttp, bioclients

Every app had the same two holes

Seven life-science Shiny apps, one author, and the same code rewritten in each.

7

APPS

7

COPIES OF "IS THIS A
GENE?"

5

HAND ROLLED API CLIENTS FOR THE SAME
SERVICE

0

THAT AGREED WITH EACH
OTHER

Bad ids got through

MLL passed every regex. It was retired in
2013.

Outages became bugs

A 503 and "no such gene" both returned
NULL.

That is not bad luck. That is a missing layer.

The tryCatch soup

```
fetch_gene <- function(symbol) {  
  url <- paste0("https://mygene.info/v3/query?q=", symbol)  
  tryCatch(  
    {  
      body <- jsonlite::fromJSON(url)  
      if (length(body$hits) == 0) return(NULL)  
      body$hits[1, ]  
    },  
    error = function(e) NULL  
  )  
}
```

no such gene

NULL

network down

NULL

HTTP 503

NULL

body was HTML

NULL

NULL erases why. A cache of NULLs poisons itself.

Four walls

Bad ids get through

Shape is not existence. Nobody suggests a fix.

`validate before you fetch`

Outages crash apps

Or worse, they look like empty results.

`a failure is a value`

Rate limits look like bugs

429s retried in a tight loop, keys pasted into URLs.

`limits belong in the client`

Nothing reproduces

The answer depends on the day, the network, and the cache.

`pin, cache, record`

Each wall became a design rule, and each rule became a package.

Three packages, one line each

biobouncer

Is this even an identifier? 50 sources, 4 modes, offline first. R, Python, JavaScript.

biohttp

Move the bytes and say what happened. One envelope, seven statuses, a cache that stores only successes.

bioclients

29 biological web services on biohttp. One table shape, one row per input.

biobouncer 0.2.0: CRAN, PyPI

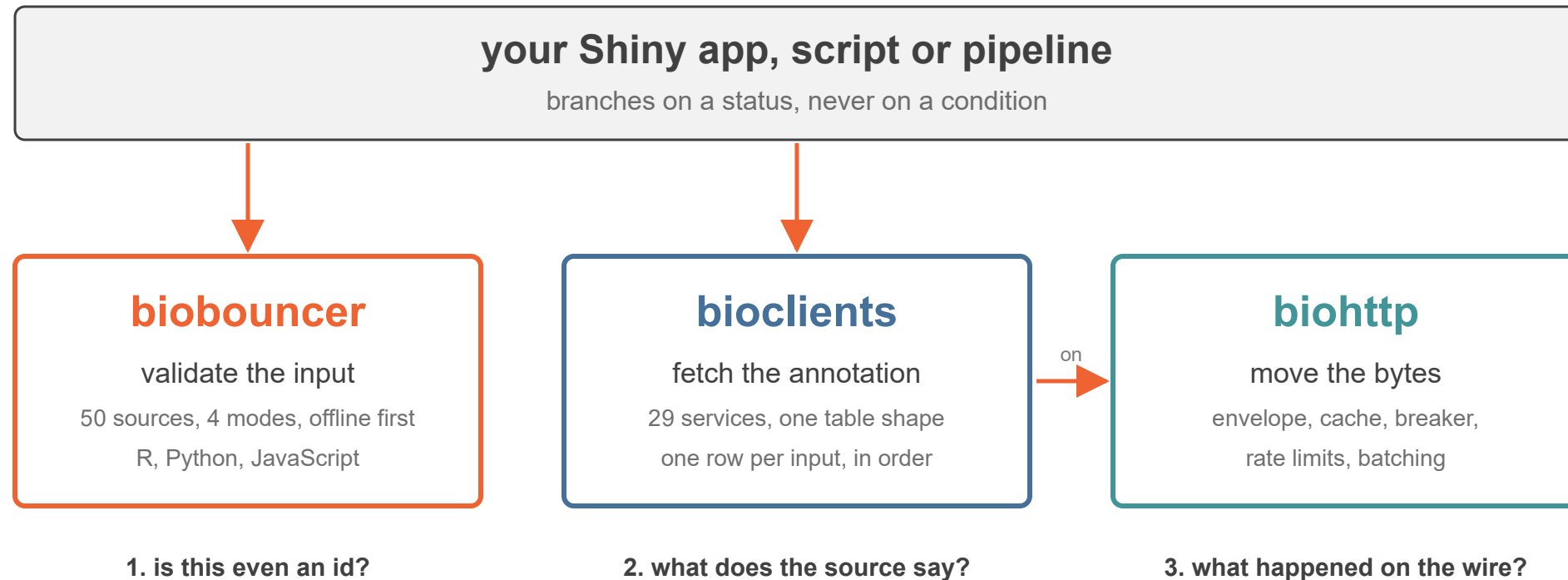
biohttp 0.1.2: CRAN

biohttp 0.1.3: r-universe

bioclients 0.1.1: r-universe

Validate, then transport, then fetch. Each one is separable, and none depends on Shiny.

The family



biobouncer is a sibling, not a dependency: it validates before either of the others is called



Is this even an identifier?

50

IDENTIFIER SOURCES

4

MODES

3

LANGUAGES, ONE CORPUS

```
check_id(c("MONDO:0005148", "mondo:5148", "GO:0006915"), "mondo")
```

#>	input	valid	suggestion
#>	MONDO:0005148	TRUE	NA
#>	mondo:5148	FALSE	MONDO:0005148
#>	GO:0006915	FALSE	NA

pattern: is it well formed?

cache: does it exist, in a dated snapshot?

remote: does it exist right now?

Offline by default. One row per input. The suggestion is what the user meant.

Repair, do not drop

```
df$gene
#> [1] "TP53" "MLL" "CARS" "BRCA2" "EGFR" "notagene" NA

report_id(df$gene, "hgnc", how = "cache")
#> 4 valid, 2 repairable, 1 invalid, 1 missing

repair_id(df$gene, "hgnc", how = "cache")
#> [1] "TP53" "KMT2A" "CARS1" "BRCA2" "EGFR" "notagene" NA
```

MLL

retired 2013, now KMT2A

CARS

→renamed, now CARS1

notagene

→unchanged. Nothing honest to replace it with.

biobouncer_snapshots()

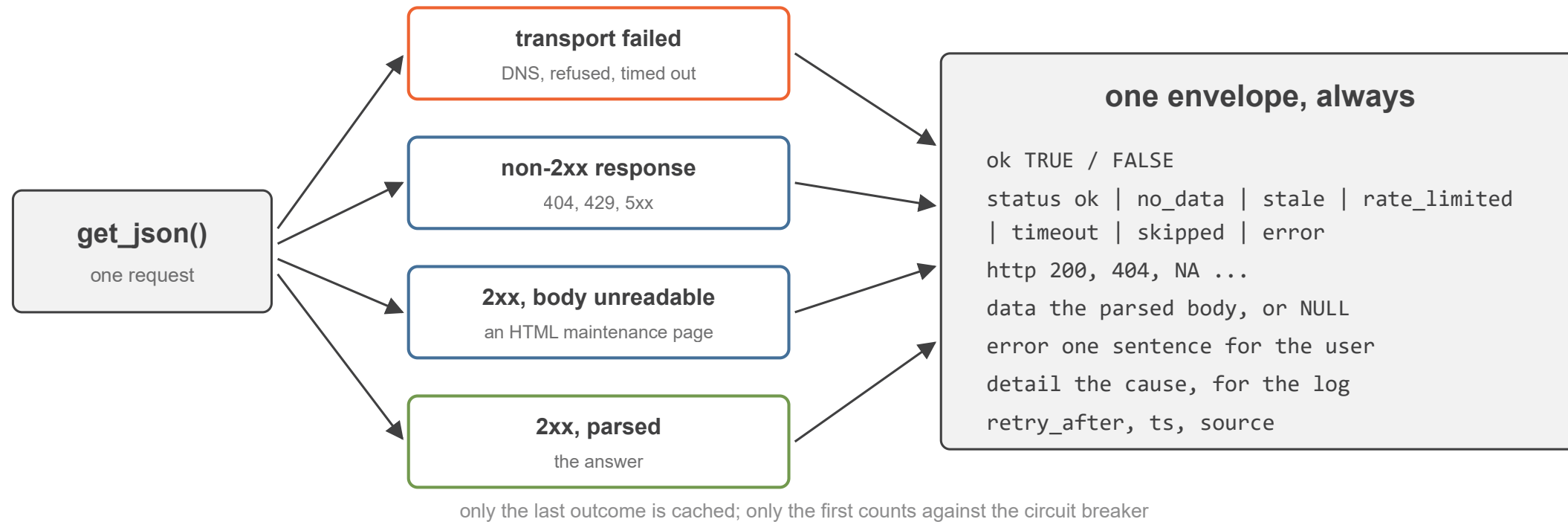
→HGNC 2026-07-07. Write it down.

The repair comes from a dated snapshot, not from a guess.



A call returns a value

It never raises. Branch on a status, not on a condition.



Seven statuses

ok

no_data

stale

rate_limited

timeout

skipped

error

```
switch(res$status,
  ok          = render_table(res$data),
  no_data     = "Nothing found for that query.",
  skipped     = "Source paused, try again shortly.",
  rate_limited = paste("Slow down, retry in", res$retry_after, "s"),
  res$error
)
```

no_data

An answer. The source was reached and has nothing.

skipped

Nothing was sent. Evidence about the host, not the query.

error, timeout

The source did not answer. Say “try again”, not “not found”.

The soup collapsed three of these into one NULL. The UI was lying during every outage.

The breaker rule

“Only a transport failure counts against a host. Any HTTP response at all, including a 500 and including a 200 whose body will not parse, proves the host is reachable and clears the count.”

from the biohttp vignette

3

FAILURES TO OPEN

120

SECONDS COOLDOWN

0

REQUESTS WHILE OPEN

A 503 proves the host is up. A hand rolled breaker that counts it turns a partial outage into a total one.



29 services, two halves each

29

SERVICES

134

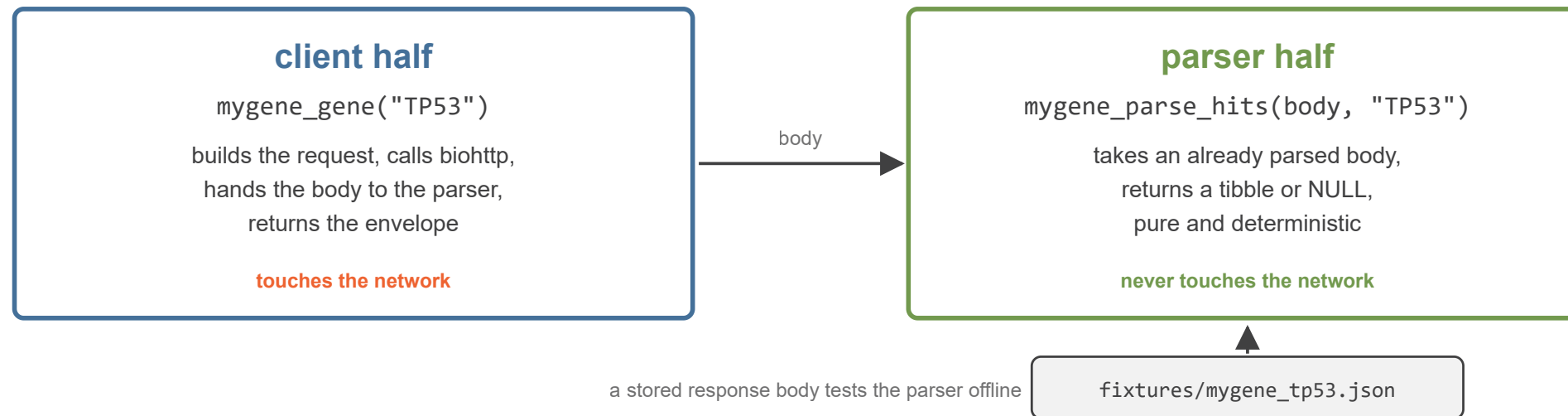
EXPORTED FUNCTIONS

57

STORED FIXTURES

0

SHINY DEPENDENCIES



The parser is pure and fixture tested. The suite runs without a network.

Reproducible by construction

offline modes

pattern and cache
never touch the
network

`how = "cache"`

dated snapshots

the version is a
string you can cite

`version = "2026-07-07"`

success-only disk cache

fixed salt, long TTL,
committed to the
repo

`BIOHTTP_CACHE_DIR`

statuses, not exceptions

an outage is a
column value in the
output

`res$status`

Four mechanisms. Each one is a file or a string you can commit.

Pipeline: eleven symbols, one table

```
genes <- read.csv("data/genes.csv")           # TP53 BRAF NF1 ... MLL CARS TP53_HUMAN NA
chk    <- check_id(genes$symbol, "hgnc", how = "cache")
clean  <- ifelse(chk$valid, chk$normalized, chk$suggestion) # 9 survive
clean  <- clean[!is.na(clean)]

ids    <- mygene_genes(clean)                  # one POST, 9 rows
cons   <- gnomad_constraints(clean)            # one GraphQL request, 9 rows
ot     <- lapply(ids$data$ensembl_gene,
                \((id) opentargets_gene_diseases(id, size = 3))

genes |>
  left_join(ids$data, by = c(clean = "symbol")) |>
  left_join(cons$data, by = c(clean = "symbol"))
```

The whole chain fits on one slide because none of it is error handling.

Second run

11

REQUESTS, FIRST RUN

0

REQUESTS, SECOND RUN

0.05 s

SECOND RUN, ALL THREE SERVICES

```
transport_stats()  
#> [1] host dispatched retried rate_limited  
#> <0 rows>  
  
identical(cons$data, cons2$data)  
#> [1] TRUE
```

Cache hit evidence is a number, not a feeling.

Pull the plug

3

TRANSPORT FAILURES RECORDED

1

STATUS: SKIPPED

0

CRASHES

```
for (i in 1:3) breaker_record("mygene.info", reachable = FALSE)

down <- mygene_genes(c(clean, "ABL1"))
down$status
#> [1] "skipped"
down$error
#> [1] "MyGene was skipped (host temporarily unreachable)."
```

The table still builds. The MyGene columns are NA and a status column says why.

The Shiny app

bio-packages: validate, fetch, branch

Gene symbols, one per line

TP53
BRAF
MLL
TP35

Validation mode

☐ pattern (shape only)

☒ cache (dated snapshot)

Validate and fetch

☐ Simulate an outage

Opens the circuit breaker for MyGene and gnomAD and bypasses the cache, so every fetch comes back skipped. Nothing crashes.

1. Validation (biobouncer)

input	valid	use	note
TP53	TRUE	TP53	
BRAF	TRUE	BRAF	
MLL	FALSE	KMT2A	repaired
TP35	FALSE	API5	repaired

3. Gene identity (bioclients: MyGene)

symbol	name	entrez	ensembl_gene
TP53	tumor protein p53	7157	ENSG00000141510
BRAF	B-Raf proto-oncogene, serine/threonine kinase	673	ENSG00000157764
KMT2A	lysine methyltransferase 2A	4297	ENSG00000118058
API5	apoptosis	8539	ENSG00000166181

2. Transport status (biohttp envelopes)

MyGene
ok
4 rows

gnomAD
ok
4 rows

4. Constraint (bioclients: gnomAD)

symbol	pli	loef	mis_z
TP53	1.00	0.42	1.12
BRAF	1.00	0.23	5.52
KMT2A	1.00	0.07	9.62
API5	1.00	0.16	4.84

Transport lab: a local server that misbehaves on demand

429 rate limited

Timeout

500 error

Press a button. The envelope comes back as a value, never a condition.

Same app, the network gone

bio-packages: validate, fetch, branch

<

Gene symbols, one per line

TP53

BRAF

MLL

TP35

Validation mode

☐ pattern (shape only)

☒ cache (dated snapshot)

Validate and fetch

☒ Simulate an outage

Opens the circuit breaker for MyGene and gnomAD and bypasses the cache, so every fetch comes back skipped. Nothing crashes.

1. Validation (biobouncer)

input	valid	use	note
TP53	TRUE	TP53	
BRAF	TRUE	BRAF	
MLL	FALSE	KMT2A	repaired
TP35	FALSE	API5	repaired

2. Transport status (biohttp envelopes)

MyGene

skipped

Source paused (breaker open)

gnomAD

ok

4 rows

3. Gene identity (bioclients: MyGene)

status	message
skipped	MyGene was skipped (host temporarily unreachable).

4. Constraint (bioclients: gnomAD)

symbol	pli	loeu	mis_z
TP53	NA	NA	NA
BRAF	NA	NA	NA
KMT2A	NA	NA	NA
API5	NA	NA	NA

Transport lab: a local server that misbehaves on demand

429 rate limited

Timeout

500 error

Press a button. The envelope comes back as a value, never a condition.

What is on CRAN and what is not

CRAN, PyPI

biobouncer 0.2.0

```
install.packages("biobouncer")
```

```
pip install biobouncer
```

CRAN 0.1.2

r-universe 0.1.3

biohttp

```
install.packages("biohttp")
```

0.1.3 adds `retry_after`,
batched retry,
`transport_stats()`.

r-universe

bioclients 0.1.1

```
pak::pak(  
  "samuelbharti/bioclients/pkg-r")
```

The `/pkg-r` suffix is
required.

Say the versions once, out loud, and nobody installs the wrong one.

Start here

Tonight

```
install.packages(c("biobouncer", "biohttp"))
install.packages("bioclients",
  repos = "https://samuelbharti.r-universe.dev")

library(biobouncer); library(bioclients)
clean <- repair_id(c("MLL", "TP53", "notagene"), "hgnc", how
mygene_genes(clean)$status
```

This talk

- Four tutorials, one per package and one pipeline, all render offline
- A Shiny app under 200 lines
- A committed transport cache and the scripts that warm and verify it

Docs hub: samuelbharti.com/biobouncer,
[/biohttp](https://samuelbharti.com/biohttp/), [/bioclients](https://samuelbharti.com/bioclients/)

Validate, then fetch, then branch on what came back.

Questions

